

TaxDesk Technical Documentation

This document explains the whole project from the ground up. It assumes you can read code but not that you already know databases, web servers, or the engineering ideas behind them. Every concept is defined the first time it appears, then shown in the actual code that uses it.

Everything here describes the repository as it stands today. Where a thing is not built yet, this document says so plainly rather than describing an intention as if it were real.

How to read this. Parts 1 and 2 are the product and the shape of the system. Part 3 is the concept teaching, the longest part. Parts 4 through 6 are the data model, every function explained, and traces of what actually happens at runtime. Parts 7 through 9 are the decisions behind the code and the honest gaps. Part 10 is how to run it.

Part 1, The Product

The problem

A solo Indian tax practitioner files the same four government returns for many clients, every month. Those four are GSTR-3B, GSTR-1, EPF, and ESI. Today the tracking happens in an Excel sheet with hand ticks. Three things go wrong with that.

- ticks get forgotten, and a missed filing means a government penalty with a client's name on it
- the sheet drifts from reality, because nothing stops a wrong or duplicated entry
- the proof files, the challans and acknowledgements, sit in client folders on disk with no connection to the ticks

What TaxDesk is

A small web application that runs on the practitioner's own computer, stores everything in one local file, and answers one question every morning. Which clients still owe GSTR-3B, GSTR-1, EPF, or ESI work this month, and where is the proof for the ones already done.

The phrase for this shape is local first. There is no cloud account, no server rented somewhere, and no copy of client data leaving the machine. The practitioner opens a browser pointed at their own computer.

What is built today

- the database, six tables with the rules enforced inside the database itself
- a migration system, so the structure can change over time without anyone editing a live database by hand
- a development seed, fake data for building against
- the web application skeleton, running on localhost
- onboarding, which turns folders on disk into client records

What is not built yet

Service assignment per client, periods, task generation, the dashboard, document search, and the proof scanner. Part 8 explains why each is absent rather than forgotten.

Part 2, The Shape Of The System

Why not just a spreadsheet

Start from the failure. A spreadsheet cannot refuse a bad entry. You can type the same client twice, tick a filing that does not exist, write "Done" in a column that means something else, or delete a row by accident. Nothing objects.

A database can refuse. That single difference is the reason this project exists as software rather than a better spreadsheet template. Rules that must never break are written into the database, and the database enforces them on every write, no matter which piece of code is doing the writing.

Why the code is in layers

Imagine all the code in one file. A page that draws HTML also runs database commands, and a change to a table breaks the HTML. Worse, a page meant only to display something could accidentally write data, because nothing separates the two jobs.

Splitting the work into layers means each layer has one job and talks only to the layer below it.

```
browser
|
web layer    HTTP, requests and responses
|
data access  app/routes/, parses input, decides what to show, redirects
|            plain Python function calls
SQLite       database/queries.py, every SQL statement lives here
|            SQL over one connection per request
disk         one local file, enforces the data rules itself
|
disk         the practitioner's client folders, read, never written
```

Two rules hold this together.

- routes never contain SQL, they call functions in `database/queries.py`
- the database layer never knows a web application exists, it takes a connection and returns rows

The payoff is that either half can change without touching the other, and each half can be tested on its own.

What each directory is for

app/	the web application
main.py	builds the application, runs migrations at startup
deps.py	shared pieces, the template engine and the connection
routes/	one file per group of pages
templates/	the HTML
database/	everything about stored data
migrations/	numbered SQL files, the structure
migrate.py	opens connections, applies migrations
queries.py	every SQL statement the application runs
seed.py	fake data for development only
docs/	this document and its siblings
tests/	the automated checks

Part 3, The Fundamentals

This part defines every engineering idea the project uses. Each one follows the same shape. What problem it solves, what it actually is, and where it appears in this code.

3.1 What a relational database is

The problem. Facts need somewhere to live that survives the program closing, and those facts have relationships to each other.

What it is. A store of tables. A table is a grid. Each column is one kind of fact, such as a name. Each row is one thing, such as one client. The word relational refers to rows in one table pointing at rows in another.

In TaxDesk. Six tables, described in Part 4. Clients point at the services they need, tasks point at a client and a service and a month.

3.2 What SQL is

The problem. Something must express what you want from the tables.

What it is. A language for talking to a relational database. Four statements carry almost everything in this project.

- `SELECT`: read rows
- `INSERT`: add a row
- `UPDATE`: change existing rows
- `DELETE`: remove rows, which this project deliberately almost never uses, see 3.12

In TaxDesk. Every SQL statement the running application uses is in `database/queries.py`. The structure statements, the `CREATE TABLE` ones, are in `database/migrations/`.

3.3 Primary keys

The problem. You need a way to point at exactly one row and never accidentally mean a different one.

What it is. A primary key is a column, or a group of columns, whose value identifies a row uniquely and never repeats in that table.

Why not use the client's name as the key. Names get corrected, respelled, and reused. If everything pointed at the name, then fixing a typo would break every pointer. A number that never changes is safer, so the row keeps a plain integer id whose only job is identity.

In TaxDesk. `CLIENTS.ID`, `SERVICES.ID`, `TASKS.ID` are integer primary keys. `SETTINGS.ID` is an integer too, with a twist explained in 3.6.

3.4 Foreign keys and referential integrity

The problem. A task belongs to a client. Nothing should let a task exist for a client who was never created.

What it is. A foreign key is a column holding another table's primary key. Referential integrity is the guarantee that such a pointer always points at something real. The database refuses writes that would break it.

In TaxDesk. `CLIENT_SERVICES.CLIENT_ID` points at `CLIENTS.ID`, `TASKS.SERVICE_ID` points at `SERVICES.ID`, and so on. A test proves the refusal is real, an attempt to insert a document for a nonexistent client raises an error.

3.5 Composite keys, surrogate keys, natural keys

Three related ideas, easiest side by side.

- natural key: the real world fact already identifies the row. A month is identified by its year and its month number, nothing else is needed
- surrogate key: an invented id with no business meaning, used when the natural identity is awkward to carry around
- composite key: a key made of more than one column

In TaxDesk, all three appear.

- `PERIODS` uses a natural composite key, `(YEAR, MONTH)`. August 2026 can exist only once, and the key says so directly
- `CLIENT_SERVICES` uses a composite key, `(CLIENT_ID, SERVICE_ID)`. The pair is the fact, so the pair is the identity
- `TASKS` uses a surrogate key. Its natural identity is four columns, client and service and year and month, which is clumsy for other tables to reference, so it carries a plain `ID` and keeps the natural identity as a separate rule, see 3.7

3.6 Constraints, rules the database enforces

The problem. A rule written only in application code holds until one code path forgets it. Six months later a different function writes the same table and the rule quietly stops applying.

What it is. A constraint is a rule attached to the table. The database checks it on every write from anyone, forever.

The kinds this project uses.

- NOT NULL: this column must have a value
- UNIQUE: no two rows may repeat this value or combination
- CHECK: the value must satisfy a condition
- PRIMARY KEY: identity, and implicitly unique and not null
- FOREIGN KEY: must point at an existing row

In TaxDesk, the important ones.

- `UNIQUE (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)` on `TASKS` . This is the single most important line in the schema. It makes a duplicate task physically impossible, so the central promise of the product does not depend on the code being correct
- `FOLDER_PATH TEXT NOT NULL UNIQUE` on `CLIENTS` . Two clients can never claim the same folder
- `CHECK (STATUS IN ('pending', 'done', 'not_applicable'))` on `TASKS` . A typo such as "Done" is rejected at write time
- `CHECK (ID = 1)` on `SETTINGS` . A clever use, explained in Part 4, it caps the whole table at one row

3.7 Normalization

The problem. If the same fact is written in two places, the two copies drift apart, and then nobody knows which is true.

What it is. Organising tables so each fact lives in exactly one place, and everything else points at it.

In TaxDesk. A service name such as GSTR-3B is stored once, as a row in `SERVICES` . Tasks store the service id, never the text. Renaming a service later would touch one row, and nothing could disagree, because nothing else holds a copy.

The test for whether a design is normalized enough. Changing one real world fact should change one row.

3.8 Many to many, and the junction table

The problem. One client needs several services. One service applies to many clients. Neither table can hold that on its own, because a column can hold one value, not a list.

What it is. A third table whose rows are the pairings. It is called a junction table, or a join table.

In TaxDesk. `CLIENT_SERVICES` holds one row per client and service pair, keyed by the pair itself so the same pairing cannot exist twice.

3.9 Soft deletion, keeping history

The problem. When a practitioner stops filing EPF for a client, the past still happened. Tasks were generated, some were completed. Deleting the relationship would erase the explanation for that history.

What it is. Instead of removing the row, a flag marks it inactive. The record survives, the behavior changes.

In TaxDesk. `CLIENT_SERVICES.ACTIVE` is that flag, and the schema file says so in a comment, off is a flag, never a delete. Task generation reads only rows where `ACTIVE = 1` .

3.10 Parameterized queries and SQL injection

The problem. If user input is glued into a SQL string, the input can change the meaning of the command. A field containing `' ; DROP TABLE CLIENTS; --` stops being data and becomes an instruction. That attack is called SQL injection.

What it is. A parameterized query sends the SQL and the values separately. The value is filled in after the command is parsed, so it can never be read as part of the command.

In TaxDesk. Every statement uses `?` placeholders and passes values as a tuple. There is no string built from user input anywhere in `database/queries.py`.

```
conn.execute(
    "INSERT OR IGNORE INTO CLIENTS (NAME, FOLDER_PATH) VALUES (?, ?)",
    (name, folder_path),
)
```

3.11 Transactions, and ACID

The problem. Some changes only make sense together. The migration runner applies a schema file and then records that it did. If the program died between those two steps, the database would be wrong about itself forever.

What it is. A transaction groups writes so that either all of them become permanent or none of them do. Committing makes them permanent. Rolling back discards them.

ACID is the four letter summary of what a database transaction guarantees.

- Atomicity: all of it or none of it
- Consistency: committed data satisfies every constraint
- Isolation: concurrent work does not see each other's half finished changes
- Durability: once committed, it survives a crash

In TaxDesk, two places.

- `database/migrate.py` wraps each migration and its logbook record in one transaction, so the pair cannot separate
- `app/deps.py` commits a web request's writes only if the request finished cleanly, and rolls them back if it raised

3.12 Idempotency

The problem. Real usage is messy. Buttons get double clicked, scripts get rerun, and people forget what they already did.

What it is. An operation is idempotent when doing it twice leaves the same result as doing it once.

In TaxDesk, deliberately, everywhere it matters.

- applying migrations, the logbook makes a rerun do nothing
- the development seed, running it twice produces byte identical state, and a test proves it by comparing every row
- creating clients, `INSERT OR IGNORE` plus the `UNIQUE` rule absorbs a repeat
- task generation, the same shape, proven by the seed

3.13 SQLite, and its three traps

What it is. SQLite is a relational database that runs as a library inside your own program rather than as a separate server. The whole database is one ordinary file. For one office on one machine that is close to ideal, and its weakness, many programs writing at once, is a problem this product does not have.

Three behaviors surprised this project and are worth knowing.

- foreign keys are off by default, per connection. SQLite ships that way for backwards compatibility, and the setting cannot be stored in the file or in the schema. Every single connection must run `PRAGMA foreign_keys = ON`, which is why one function owns opening the database and nothing else may bypass it
- `executescript` commits on its own. It ends any open transaction before running, so calling `.commit()` afterwards does not make the script atomic with anything else. This project therefore writes `BEGIN` and `COMMIT` inside the executed script itself
- there are no real date or length types. `VARCHAR(100)` is accepted and the length is ignored, and dates are text you format yourself. This project stores text in ISO order and says so

3.14 Migrations

The problem. The database file holds real client data and never leaves the practitioner's machine, so it cannot be shipped. The structure, though, must be identical on every machine, and must be able to change over time without anyone editing a live database by hand.

What it is. Each structural change is a numbered SQL file, committed to version control. A runner applies the files a given database has not seen yet, in order, and records each one it applied. Applied files are then frozen forever, because editing one would never reach databases that already ran it. A change is always a new file.

In TaxDesk. `database/migrations/001_schema.sql` and `002_settings.sql`, applied by `initialize()` in `database/migrate.py`, recorded in a table named `schema_applied` that lives inside the database itself so each database knows its own state.

3.15 HTTP, requests and responses

What it is. The language a browser and a server speak. The browser sends a request, the server sends back a response. Nothing is remembered between requests unless something deliberately stores it, which is what the word stateless means.

The pieces used here.

- methods, the verb of the request. GET means read me something, POST means here is data to save
- status codes, the three digit verdict. 200 fine, 302 and 303 go to another page, 400 your input was bad, 404 no such thing, 500 the server broke
- the body, the form fields on the way in, the HTML on the way out

Why the GET and POST distinction matters more than it looks. A browser will repeat a GET freely, on refresh, on back, on a prefetch. If a write hid behind a GET, a refresh would silently write again. So reads are GET, writes are POST, with no exceptions.

3.16 The post redirect get pattern

The problem. After a form POST, the browser is sitting on the result of a write. Pressing refresh resubmits the form and writes twice.

What it is. The server answers a successful POST with a redirect to a normal page. The browser then performs a plain GET, and that GET is what the user is sitting on. Refresh now repeats the harmless read.

In TaxDesk. Every POST route ends with a 303 redirect. The 303 code specifically instructs the browser to use GET for the next request, which is the detail that makes the pattern work.

3.17 Routing, and what a decorator really does

What it is. Routing is the map from a method and a path, such as GET and `/onboarding`, to the function that answers it.

The decorator. A line like `@router.get("/onboarding")` above a function is a decorator. It is not magic. It is a function that receives the function written below it and, in this case, stores an entry in a table saying that GET on that path is handled here. This happens once, when the module is imported, before any request arrives. At request time the framework just looks the path up.

In TaxDesk. Five routes exist today.

```
GET /                redirect to onboarding
GET /health          liveness, returns {"status": "ok"}
GET /onboarding       the root form plus folder discovery
POST /onboarding/root save the root folder
POST /onboarding/confirm create clients from confirmed folders
```

3.18 The server and the framework are two different things

A common confusion worth clearing. FastAPI decides what to answer. Uvicorn is the program that actually listens on a network port, accepts connections, and hands each request to FastAPI. FastAPI cannot listen on its own, and uvicorn does not know what your routes mean. They are separate jobs, and the interface between them is a standard called ASGI.

3.19 Dependency injection

The problem. Every route needs a database connection, opened correctly, closed reliably, committed on success and rolled back on failure. Twenty routes each doing that by hand means twenty chances to get it wrong.

What it is. The route declares what it needs rather than building it. The framework supplies it. That is the whole idea, receive your tools, do not construct them.

In TaxDesk. A route signature says `conn: Connection = Depends(get_db)`. Before the route runs, FastAPI calls `get_db` in `app/deps.py`. That function is a generator, it opens the connection and pauses at `yield`, handing the connection over. When the route finishes, the function resumes and either commits or rolls back, then always closes. One

correct lifecycle, written once, impossible for a route to forget.

3.20 Templates and automatic escaping

The problem. Building HTML by joining strings is unreadable, and dangerous. If a client were named `<script>steal()</script>` and that text were pasted into the page, the browser would run it. That attack is called cross site scripting, XSS.

What it is. A template is HTML with labelled blanks. The engine fills the blanks and escapes the values on the way in, turning `<` into `<`, so a value can never turn into code.

In TaxDesk. Jinja2 templates in `app/templates/`. `base.html` is the shared frame, other pages extend it. Escaping is on by default, so safety is the default rather than something to remember.

3.21 Trust boundaries

The problem. Data arriving from outside the program may be shaped however the sender wants, including deliberately hostile shapes.

What it is. A trust boundary is the line where outside data enters. At that line, the program validates rather than assumes. Running on localhost does not remove this, because the browser is still outside the program.

In TaxDesk, the sharpest example. The confirm step of onboarding never trusts submitted folder names. It rescans the real root folder, builds the set of names that genuinely exist, and accepts only names in that set. A crafted submission such as `../evil`, or an absolute path pointing somewhere else entirely, matches nothing and is silently ignored. A test sends exactly those and proves no client is created.

That style is called deriving the valid set from a trusted source rather than validating the untrusted input, and it is the stronger of the two approaches, because it cannot be fooled by an input shape nobody predicted.

3.22 Testing

Why. Manual checking decays. Code changes, and behavior that worked yesterday quietly breaks with nothing to notice.

The kinds used here.

- unit test: exercises one function directly
- integration test: exercises several layers together, for example a real HTTP request that reaches the real database
- fixture: shared setup a test asks for by name, such as a fresh database

A rule this project follows. A test must be able to fail. A test that passes no matter what the code does proves nothing. When the rollback behavior in `get_db` had no test, that behavior was unproven even though it was written and documented, so a test was added that writes and then deliberately raises.

In TaxDesk. 33 tests, all on temporary databases in temporary folders, so the developer's real database and real files are never touched.

tests/test_database.py	8	migrations, constraints, rollback boundary
tests/test_seed.py	5	seed content and repeatability
tests/test_app.py	7	skeleton, health, startup, transactions
tests/test_onboarding.py	13	discovery, confirmation, trust boundary

3.23 Version control, branches, and review

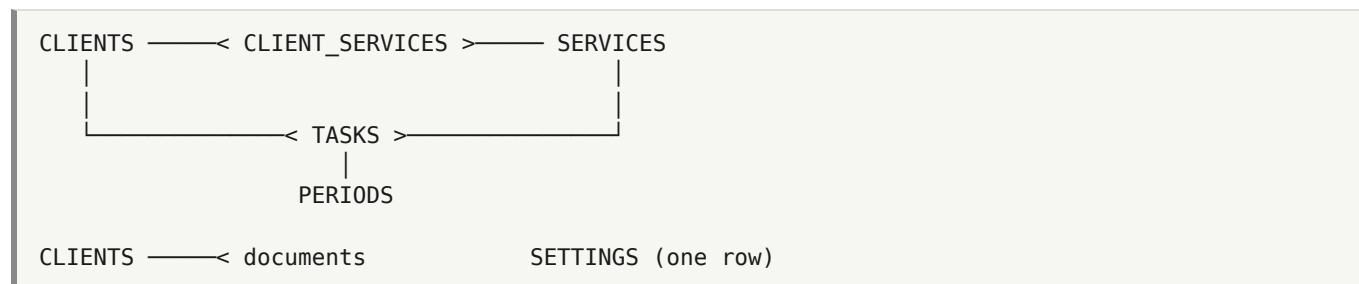
What it is. Git records snapshots of the project called commits. A branch is a line of work separate from the main line. A pull request, PR, proposes merging a branch and is where review happens.

The workflow this project uses. One branch per intent, named for it. Work is committed and pushed there. A PR is opened. Review happens on the PR, findings get fixed on the same branch, and only then does it merge into main. Main therefore only ever contains reviewed code.

This has already paid for itself. Three real defects were caught in review rather than in use, described in Part 7.

Part 4, The Data Model

Six tables. The full definitions are in `database/migrations/001_schema.sql` and `002_settings.sql`.



CLIENTS

Who the practitioner works for. Holds the name and `FOLDER_PATH`, the absolute path of that client's folder on disk. The path is **UNIQUE**, because a folder identifies a client and two clients cannot share one.

SERVICES

One row per filing type. The four rows, GSTR-3B, GSTR-1, EPF, ESI, are inserted by the migration runner rather than by the schema, for a reason explained in Part 7. The name is **UNIQUE**.

This being a table rather than four fixed columns is what makes a fifth filing type a new row someday instead of a structural change.

CLIENT_SERVICES

The junction table from 3.8. One row per client and service pair, keyed by the pair, carrying `ACTIVE` as the soft delete flag from 3.9.

Today nothing in the application writes to this table. Onboarding creates clients and stops. That is the gap the next feature fills.

PERIODS

One row per tracked month, keyed naturally by `(YEAR, MONTH)`, with `STATE` holding open or closed. A closed month is meant to refuse further changes, and the schema stores the state, while the rule that enforces it belongs to the application code that does not exist yet.

TASKS

The heart of the eventual product. One row means one client owes one filing for one month. It carries the status, the due date, and the completion trace, when and how a task was completed.

Its four column UNIQUE constraint is the reason duplicate work items cannot exist. Its due date is deliberately nullable, because the real due day for each filing type is not yet known and inventing one would be inventing a business rule.

documents

File links owned by a client. Deliberately minimal. The relationship between a document and a task is unresolved on purpose, because no requirement has yet forced its shape.

SETTINGS

Application configuration as exactly one row.

```
CREATE TABLE SETTINGS (  
    ID INTEGER PRIMARY KEY CHECK (ID = 1),  
    ROOT_FOLDER TEXT NOT NULL  
);
```

The `CHECK (ID = 1)` is worth pausing on. Since the primary key must be unique and this check forces it to be the value 1, the table can physically never hold a second row. No row at all means the application is not configured yet, which is how onboarding knows to ask.

Part 5, The Code, Function By Function

This part walks every function in the project. Each one gets the same three questions, why it exists, what it does, and how it works, with the lines that are not obvious explained directly.

5.1 database/migrations/001_schema.sql and 002_settings.sql

Not functions, but the foundation everything else stands on. `001` creates the six original tables with every constraint. `002` adds `SETTINGS`. Both are frozen because real databases have applied them, so future changes are new numbered files. They are executed by the runner, never by hand.

5.2 database/migrate.py

The bottom of the stack. It imports nothing from this project, only Python's standard library.

`connect(db_path)`

```
def connect(db_path: Path = DEFAULT_DB_PATH) -> sqlite3.Connection:
    conn = sqlite3.connect(db_path, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

Why. Three settings must be true on every single connection, and a human will eventually forget one. Making this the only door means nobody can forget.

What. Opens the database file, creating it if absent, and returns a connection ready to use.

How, line by line.

- `sqlite3.connect(db_path)` opens the file. In SQLite, opening and creating are the same act, so a missing file becomes a new empty database
- `check_same_thread=False` allows the connection to be used from a different thread than the one that created it. This is normally dangerous, and it is safe here only because of a rule the project holds elsewhere, one connection serves exactly one request and is then closed. FastAPI may run a request's code on different thread pool threads, which is why the default setting breaks
- `row_factory = sqlite3.Row` makes rows readable by column name, `row["NAME"]`, instead of only by numeric position. Position based access still works, so older code was not broken by adding this
- the PRAGMA turns on foreign key enforcement, which SQLite leaves off per connection

Called by. Everything, the application startup, the request dependency, the seed, every test.

`apply_one(conn, migration)`

Why. A migration and the record saying it ran must both happen or neither. If they separate, the database becomes wrong about itself, and the next run either reapplies a migration onto existing tables and crashes, or skips one that never actually ran.

What. Applies one migration file and writes its logbook row, as a single atomic unit.

How, and this is the subtle one.

```
script = (
    "BEGIN;\n"
    f"{body}\n"
    f"INSERT INTO schema_applied (filename) VALUES ('{migration.name}');\n"
    "COMMIT;"
)
conn.executescript(script)
```

The obvious implementation would run the file, then insert the record, then call `.commit()`. That was the first version and it was wrong. `executescript` commits on its own before it runs, so the `.commit()` afterwards had nothing to do with the migration, and the two halves were never joined. The fix is to put `BEGIN` and `COMMIT` inside the script itself, making one real SQLite transaction that spans both. That is why migration files must never contain their own transaction statements, they would collide with these.

Two supporting details.

- the filename is checked against an allowlist pattern before being put into the string, because placeholders do not work inside `executescript`. The names come from the project's own directory, so this is defense in depth rather than a live threat
- the `try` block rolls back if anything raises, then re raises, so the caller still sees the failure

initialize(conn)

Why. Something must decide which migrations a given database still needs, and guarantee the required service rows exist.

What. Brings any database up to date, and returns whether it changed anything.

How, in four steps.

1. create the `schema_applied` logbook table if it does not exist. `IF NOT EXISTS` means this is real work on a fresh database and a no operation on every later run
2. read the set of filenames already applied
3. loop over `sorted(MIGRATIONS_DIR.glob("*.sql"))`, skipping any name already in that set, and call `apply_one` for the rest. The sort is why filenames are numbered, `001` before `002`, so order is deterministic rather than whatever the filesystem returns
4. ensure the four required services with `INSERT OR IGNORE`, outside the migration loop, so this runs on every call. That placement is deliberate. A fifth service name added to `REQUIRED_SERVICES` later reaches databases that were initialized long ago, which would be impossible if the rows lived inside a frozen migration

Returns `True` if at least one migration was applied, `False` if the database was already current, which is what the command line block prints.

the `__main__` block

Why. So a developer can run the runner directly.

How. Takes an optional database path from the command line, otherwise uses the default, connects, initializes, prints the outcome, and closes the connection in a `finally` so it closes even on failure.

5.3 database/queries.py

Every SQL statement the running application uses. Four functions today, and only what a shipped feature needs.

`get_root_folder(conn)`

Why. Reading the configured root is the first thing the application does on almost every page.

What and how. Selects `ROOT_FOLDER` from the single settings row and returns it, or `None` when no row exists. That `None` is meaningful, it is the application's definition of not configured yet, which is how onboarding knows to ask.

`set_root_folder(conn, root_folder)`

Why. Saving the root must work whether or not a value was saved before, without the caller having to check first.

How.

```
INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (1, ?)
ON CONFLICT(ID) DO UPDATE SET ROOT_FOLDER = excluded.ROOT_FOLDER
```

This is an upsert, an insert that becomes an update when the row already exists. `excluded` is SQLite's name for the row that was being inserted, so the clause reads as use the new value. One statement handles both the first save and every later change, so there is no branch in the calling code to get wrong.

`list_clients(conn)`

Why. Two callers need every client, the onboarding page to mark folders as already added, and any future client listing.

How. A plain `SELECT` ordered by name, so the order is deterministic rather than insertion order.

`create_client(conn, name, folder_path)`

Why. Onboarding creates clients in a loop, and a repeat submission or a double click must not become an error page.

How. `INSERT OR IGNORE`, which turns a `UNIQUE` violation into a quiet no operation. The `UNIQUE` constraint on `FOLDER_PATH` is still the real guard, this just decides how the application reacts to it, which is calmly.

5.4 database/seed.py

A development tool. It fills a database with obviously fake data so pages can be built against something. It is never run on a real machine, and nothing in the application calls it.

the module constants

`SEED_CLIENTS` holds five invented clients as tuples of name, folder path, and the services each subscribes to.

`SEED_PERIOD` is August 2026. `SEED_COMPLETED_AT` is a fixed timestamp string, and that fixed value matters, see `mark_sample_statuses` below.

`service_id(conn, name)`

Why. The seed knows service names, but the tables store service ids.

How. Selects the id for a name. It deliberately does not handle a missing name, because a missing service would mean initialization never ran, and failing loudly is the correct response to that.

`seed_clients(conn)`

Why. Clients and their subscriptions are the base every other seeded row depends on.

How. For each entry, insert the client with `INSERT OR IGNORE`, read back its id by folder path, then insert a `CLIENT_SERVICES` row per subscribed service, again with `INSERT OR IGNORE`. Afterwards one subscription is switched to `ACTIVE = 0` with a targeted `UPDATE`, so the inactive state exists in development data and generation can be seen skipping it.

`seed_period_and_tasks(conn)`

Why. Tasks are what the eventual product is about, so development data needs them.

How. Insert the period, then one statement generates every task.

```
INSERT OR IGNORE INTO TASKS (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)
SELECT CLIENT_ID, SERVICE_ID, ?, ? FROM CLIENT_SERVICES WHERE ACTIVE = 1
```

This is an insert fed by a select, so the database does the whole job in one pass without any Python loop. `WHERE ACTIVE = 1` is what makes switched off subscriptions produce nothing. `INSERT OR IGNORE` plus the four column `UNIQUE` constraint is what makes running it twice harmless. This is the exact shape the real application will use when task generation is built.

`mark_sample_statuses(conn)`

Why. Every screen state needs data to show, so development data should include a completed task and a not applicable one, not only pending ones.

How. Two targeted UPDATES, selecting their targets by client name and service name rather than by hardcoded ids, so they stay correct whatever ids the database assigned.

The important detail. The completion timestamp is the fixed constant, not `datetime('now')`. The original version used the current time, which meant running the seed twice changed the data even though the row counts matched. That contradicted the promise that the seed is repeatable, and the test at the time was too weak to notice because it only compared counts.

print_summary(conn)

Why. A script that finishes silently gives no evidence it worked.

How. A GROUP BY over pending tasks joined to service names, printed as counts. It is proof of life, and it is also the first real use of the aggregation the dashboard will eventually need.

main()

How. Resolves the database path from the command line or the default, connects, calls `initialize` first so a brand new file works, runs the three seeding steps, commits once, prints the summary, and closes in a `finally`.

5.5 app/main.py

lifespan(application)

Why. Migrations must be applied before the first request is served, so installing an update and opening the application is all anyone ever has to do.

How. An async context manager wired into FastAPI's startup and shutdown. Everything before `yield` runs at startup, everything after would run at shutdown, and there is nothing after because there is nothing to clean up. It opens a connection, calls `initialize`, and closes that connection immediately in a `finally`. Requests never touch this connection, they each get their own.

create_app(db_path)

Why. Tests need a real application pointed at a temporary database, and the alternative would be an environment variable configuration system nobody needs yet.

How. Builds the `FastAPI` object with the lifespan attached, stores the database path on `application.state`, includes both routers, and returns it. Storing the path on the application is what lets `get_db` find it later through the request.

app = create_app()

The production instance, built with the default path. This is the object the command `uvicorn app.main:app` loads, the `app` in that string is literally this variable.

5.6 app/deps.py

templates

The Jinja2 engine, pointed at `app/templates/`, created once at import rather than per request.

get_db(request)

Why. The connection lifecycle is easy to get wrong and needs to be identical everywhere.

```
def get_db(request: Request) -> Iterator[Connection]:
    conn = connect(request.app.state.db_path)
    try:
        yield conn
        conn.commit()
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()
```

How, and the control flow here is worth reading slowly. This is a generator, not a normal function. FastAPI calls it, it runs down to `yield`, and pauses there, handing the connection to the route.

- if the route returns normally, execution resumes just after the `yield`, so `conn.commit()` runs and every write the route made becomes permanent together
- if the route raises, the exception is thrown back into the generator at the `yield` line, the `except` catches it, rolls back every write, and re raises so FastAPI still turns it into a 500
- `finally` closes the connection either way

Why `BaseException` rather than `Exception`. When a client disconnects mid request, Python closes the generator by throwing `GeneratorExit`, which is not an `Exception`. Catching the broader type means the rollback still happens, and re raising is required because a generator that swallows `GeneratorExit` raises a `RuntimeError`.

This function is the reason routes contain no transaction code at all.

5.7 app/routes/health.py

health(conn)

Why. One endpoint that proves the entire spine works, uvicorn to FastAPI to the dependency to SQLite and back.

How. Runs `SELECT 1` through the injected connection, discards the result, and returns the dictionary `{"status": "ok"}`. The query result is deliberately not exposed, and the response deliberately carries nothing else. A test asserts key for key equality so no version, path, or internal detail can quietly join it later.

5.8 app/routes/onboarding.py

The filesystem is read only in this entire module. Nothing here creates, renames, moves, or deletes anything on disk.

candidate_folders(root_folder)

Why. Both the page and the confirm step need the same answer to one question, which folders inside the root are real candidates.

How.

```
root = Path(root_folder)
if not root.is_dir():
    return []

try:
    children = list(root.iterdir())
except OSError:
    return []

return sorted(
    (child for child in children
     if child.is_dir() and not child.name.startswith(".")),
    key=lambda child: child.name.lower(),
)
```

- the early return handles a root that has stopped existing
- the `try` around `iterdir` handles a subtler case found in review. A directory can stat as perfectly real while still refusing to be listed, because of a permission change or a removable drive that dropped. Without this guard that raised a `PermissionError` all the way to the browser as a 500
- `child.is_dir()` drops plain files, and the `startswith(".")` test drops hidden folders
- there is no recursion anywhere, only immediate children
- sorting by lowercased name makes the page order stable and human friendly rather than filesystem order

discovery_context(conn, root_folder)

Why. The page and the error path of `save_root` both need the same block of template values, and computing it twice invited them to drift apart, which is exactly what happened before review caught it.

How. Returns `None` candidates when no root is configured. Otherwise it builds the set of folder paths already registered as clients, then one dictionary per candidate folder carrying its name and whether it already exists as a client. It counts the new ones and formats the button label, singular or plural, so the button can say `Add 1 client` or `Add 12 clients` rather than something vague.

The comparison uses `str(folder.resolve())` against stored paths, resolved absolute paths on both sides, so the match is reliable.

home()

Why. Something must answer the bare URL.

How. Returns a 302 redirect to `/onboarding`, the only page that exists. It takes no database connection because it needs none.

onboarding_page(request, conn)

Why. The main screen, the root form plus discovery.

How. Reads the configured root, then builds the template values. Note the two names, `root_folder` fills the text input, and `configured_root` gates and labels the discovery panel. On this happy path they are the same value. They exist separately because of `save_root` below, where they legitimately differ. The `**discovery_context(...)` merges the shared block in.

save_root(request, conn)

Why. The root folder must be validated before it is stored, since every later scan depends on it.

How. Reads the form, strips whitespace, and checks the path is a real directory. On success it stores the resolved absolute path and redirects with 303. On failure it re renders the page with a 400 status and no write at all.

The failure path is more careful than it looks, and it took a review to get right. It passes the submitted value as `root_folder`, so the practitioner sees exactly what they typed and can fix the typo in place, and it passes the still valid saved root as `configured_root` along with its real discovery panel, so a rejected new path does not make a working configuration appear to vanish. The earlier version blanked the panel and echoed the old value, which was confusing in both directions.

This function is `async` because reading a form body is `await request.form()`, work that waits on the network.

confirm_clients(request, conn)

Why. This is where rows are created, so this is where the trust boundary lives.

How, and the order of operations is the whole point.

```
actual_folders = {
    folder.name: folder for folder in candidate_folders(root_folder)
}

form = await request.form()
for name in form.getlist("folders"):
    folder = actual_folders.get(str(name))
    if folder is None:
        continue
    queries.create_client(conn, name=folder.name, folder_path=str(folder.resolve()))
```

The valid set is built first, from a fresh scan of the real disk, and the submitted names are then looked up inside it. A submitted value that is not a real immediate subfolder finds nothing and is skipped. That single design decision defeats a whole family of attacks at once, `../evil`, an absolute path to somewhere else, a name that never existed, without needing to predict any of them. A test submits exactly those three and proves no client is created.

`form.getlist` is used rather than `form.get` because checkboxes sharing a name submit repeated values, and only `getlist` returns all of them.

Nothing here deletes or updates. Existing clients are untouched, and `create_client` absorbs duplicates, so pressing the button twice is harmless.

5.9 app/templates/

`base.html` is the page frame, currently a title and a content block. `onboarding.html` extends it and holds the root form, the discovery checkboxes, and the state messages, empty root, no root configured, folders found, folders already added.

One template detail worth knowing. An already added folder renders its checkbox as `disabled`, and a disabled checkbox is not submitted by the browser at all. So existing clients cannot be resubmitted even if someone tampers with the page, which is a second layer under the server side check.

Templates never call application code. They display what a route hands them.

5.10 tests/

Four files, 33 tests. Every test builds its own temporary database in a temporary folder, so no test can affect another or touch real data.

- `conftest` style fixtures appear in each file rather than a shared one, since the suites need slightly different setups
 - `test_database.py`, 8 tests, migrations applying in order, reruns doing nothing, the settings single row rule, foreign keys actually enforced, and the rollback boundary where a migration breaks halfway and must leave no trace
 - `test_seed.py`, 5 tests, the four services present after initialization, the seeded row counts, generation skipping inactive subscriptions, and a full state comparison proving a second seed run changes nothing
 - `test_app.py`, 7 tests, health returning exactly the expected body, startup applying migrations, the dependency override being used, the rollback path, a plain 404, and a leak check on the health response
 - `test_onboarding.py`, 13 tests, the whole flow including the traversal attempt, the unreadable root, the empty root, and the rule that changing the root never rewrites existing client paths
-

Part 6, What Actually Happens At Runtime

Starting the application

```
venv/bin/uvicorn app.main:app
```

1. uvicorn starts as an operating system process
2. it imports `app.main`, which imports the route modules, and the decorators register every route while importing
3. `lifespan` runs, opens the database, applies pending migrations, ensures the four services exist, closes that connection
4. uvicorn binds 127.0.0.1 port 8000 and waits

Binding to 127.0.0.1 means the operating system only accepts connections from this same machine. That is the security boundary of the product, and it is why no host flag is ever passed.

A read, GET /onboarding

1. browser sends GET /onboarding
2. uvicorn hands the request to FastAPI
3. FastAPI matches the path to `onboarding_page`
4. the signature asks for a connection, so `get_db` opens one
5. `queries.get_root_folder` reads the settings row
6. `discovery_context` lists the real subfolders and compares them against existing client paths
7. the route hands values to `onboarding.html`, Jinja2 fills and escapes them
8. the HTML travels back, `get_db` commits nothing of substance and closes the connection

A write, POST /onboarding/confirm

1. browser sends the ticked folder names
2. FastAPI calls `confirm_clients`, `get_db` opens a connection
3. the route reads the saved root, then rescans the disk to build the set of folder names that really exist
4. for each submitted name, if it is in that set, a client is created, otherwise it is silently ignored
5. the route returns 303 pointing at /onboarding
6. `get_db` commits, making every created client permanent together
7. the browser follows the redirect with a GET, so a refresh cannot resubmit

If step 4 had raised instead, `get_db` would have rolled back and no client would have been created at all, not even the ones already inserted in the loop.

Part 7, Decisions And Rejected Alternatives

Every entry here records a real choice, what it beat, and why. The running record with more detail is `docs/engineering-log.md`.

Services as a table, not fixed columns

Rejected. Four boolean columns on `CLIENTS`, or the four names baked into a `CHECK` constraint. Both make a fifth filing type a structural change. The lookup table makes it a row.

Required services live in the runner, not in the schema file

Rejected, and caught in review. The four service rows were first written as `INSERT` statements inside `001_schema.sql`. That file is applied once and then frozen forever, so a fifth service added later would never reach a database that already ran it. They moved into `REQUIRED_SERVICES` in the runner, ensured on every initialize call. The general lesson is that anything inside an applied migration is frozen the moment one real database runs it.

Settings as one typed row, not a key and value store

Rejected. A generic table of key and value pairs. There is exactly one known configuration value, so a generic store was flexibility nobody asked for, and it would have thrown away typed columns and constraints. A future second value is a migration adding a column.

Transactions inside the migration script

Rejected, and caught in review. The first version applied a migration and then recorded it, relying on one `.commit()` call. Verification showed `executescript` commits on its own, so the two were never atomic and a crash between them could leave a migration applied but unrecorded. The fix carries `BEGIN` and `COMMIT` inside the executed script. A test now breaks a migration halfway and proves neither its tables nor its record survive.

Confirm before create, in onboarding

Rejected. Automatically turning every subfolder into a client. In a system that deliberately never deletes, the moment before a row exists is the only cheap place for human judgment. Folders such as Backup and Old are not clients, and only the practitioner knows that.

Deterministic seed data

Caught in review. The seed set a completion timestamp to the current time, so a second run changed data even though the row counts matched, which contradicted the claim that it was repeatable. The timestamp became a fixed constant, and the test was strengthened from comparing counts to comparing every value of every row.

Raw SQL, no ORM

Rejected. An object relational mapper, a library that hides SQL behind objects. The project's second purpose is learning, and hidden SQL defeats that. The cost accepted is writing queries by hand, which at this size is small.

Part 8, What Is Not Built, And Why

Each of these is absent deliberately, not forgotten.

- assigning services to clients. The next feature. Until it exists, every onboarded client has zero services, so nothing downstream can produce work
 - periods and task generation. They depend on service assignment
 - the dashboard. It depends on tasks existing
 - documents and the proof scanner. They depend on knowing what real filenames look like, which requires seeing a practitioner's actual folders
 - real due dates. The due day for each filing type is a business fact the practitioner has to supply. Guessing would put invented rules into a compliance tool
 - authentication. The application listens only on localhost, so the operating system login is the front door. This must be revisited the moment anything is reachable from another machine
 - continuous integration. Nothing runs the 33 tests automatically on every change yet
 - backups. There is no backup mechanism for the database file
-

Part 9, Honest Limitations

Named plainly, because a document that only lists strengths is marketing.

- no backup exists. Once real data lives in `database/taxdesk.db`, a disk failure loses it. This is the most important gap before the product is used for real work
 - no automated test run on changes. The suite is only as useful as someone remembering to run it
 - the seed writes rows for subscribed services only, while the upcoming service assignment feature will write rows for every service including inactive ones. Two shapes for one table, worth reconciling
 - the application has no navigation. `base.html` is a heading and a content block, so features are reachable only by typing URLs
 - changing the root folder does not repair existing client paths. If a whole tree moves, stored paths go stale, and path repair is a named future feature rather than a silent side effect
 - there is no way to edit or archive a client once created
 - table naming is inconsistent, most tables are uppercase and `documents` is lowercase. Cosmetic, and frozen by applied migrations, so a rename would need a rebuild migration
-

Part 10, Running And Developing

Running the application

```
python3 -m venv venv
venv/bin/pip install fastapi uvicorn jinja2 python-multipart
venv/bin/uvicorn app.main:app
```

Then open `http://127.0.0.1:8000` in a browser. Migrations run automatically at startup. Never pass a host flag, listening on localhost only is the security boundary.

Development tasks

```
venv/bin/pip install pytest httpx2      # test tooling
venv/bin/pytest                          # run the 33 tests
venv/bin/pytest -W error                 # treat warnings as failures
python3 -m database.seed                 # fill a development database
python3 database/migrate.py              # apply migrations by hand
```

Dependencies, and why each exists

- fastapi, the web framework, routing and dependency injection
- uvicorn, the server that listens on the port
- jinja2, HTML templates with automatic escaping
- python-multipart, required by the framework to parse form posts
- pytest, the test runner, development only
- httpx2, the HTTP client the framework's test client needs, development only

Six pinned packages, no ORM, no frontend framework, no build step.

Adding a change

1. branch from main, named for the single intent
 2. write the code and the tests together
 3. run the full suite
 4. update `docs/engineering-log.md` with purpose, what changed, why this way, and how it works
 5. open a PR, get it reviewed, fix findings on the same branch
 6. merge, then delete the branch
-

Part 11, Glossary

- ACID: the four guarantees of a database transaction, atomicity, consistency, isolation, durability
- ASGI: the standard interface between a Python web framework and the server that runs it
- constraint: a rule attached to a table that the database enforces on every write
- decorator: a line above a function that hands that function to another function, used here to register routes
- dependency injection: the framework supplies what a function declares it needs, rather than the function building it
- fixture: reusable setup a test asks for by name
- foreign key: a column holding another table's primary key
- idempotent: doing it twice leaves the same result as doing it once
- junction table: a table whose rows are pairings, used for many to many relationships
- local first: the software and the data live on the user's own machine
- migration: one numbered, recorded change to database structure
- normalization: organising data so each fact lives in exactly one place
- ORM: a library that maps database rows to objects, deliberately not used here
- parameterized query: SQL sent separately from its values, so values can never become commands
- post redirect get: answering a form POST with a redirect so refresh cannot resubmit
- PRAGMA: a SQLite specific setting command, used here for foreign keys
- primary key: the column or columns identifying a row uniquely
- referential integrity: the guarantee that pointers between tables always point at real rows
- rollback: discarding a transaction's writes
- soft delete: marking a row inactive instead of removing it
- SQL injection: an attack where input is treated as command
- surrogate key: an invented id with no business meaning
- transaction: a group of writes that all succeed or all fail
- trust boundary: the line where outside data enters a program and must be validated
- upsert: an insert that becomes an update when the row already exists
- XSS: cross site scripting, an attack where input becomes executable page content